Disjoint sets

Part V Advanced Data Structures

Scope

Disjoint sets turn a partition that only merges into representative queries. We compare weighted lists and rank forests on the same merges, then prove the inverse-Ackermann sequence bound through complete level/iter potential accounting. Single-operation cost, amortized sequence cost, and implementation assumptions remain distinct.

Prerequisites

Representatives, forests, ranks, and path compression.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Compare linked-list and forest representations.
- Apply union by rank and path compression.
- Prove the inverse-Ackermann amortized bound.
- 21.1 Disjoint-set operations
- 21.2 Linked-list representation of disjoint sets
- 21.3 Disjoint-set forests
- 21.4 Analysis of union by rank with path compression

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21.1 Disjoint-set operations

The partition is the state; a representative is a handle

Every established universe element belongs to exactly one nonempty set, and these sets are pairwise disjoint. MAKE-SET(x) requires a new element and creates {x}. FIND-SET(x) requires an existing element and returns a representative of its set. UNION(x,y) merges two sets. The representative need not be the minimum key, and internal parent pointers need not be original graph edges.

Let n count MAKE-SET calls and m count all external MAKE-SET, FIND, and UNION calls, with $m \geq n$. Establish all n singletons first to fix the analysis parameters. There are at most $n-1$ successful merges, but arbitrarily many repeated UNION calls. Find both roots first; if they agree, return without LINK or a rank increase. LINK accepts **two distinct roots**.

FIND can compress parents without changing the partition. UNION only merges. Deletion, rollback, dynamic removal of graph edges, and enumeration of all members require additional structures or operations. Different sets have different representatives; all finds within one set return the same representative.

Why union maintains exactly the connected components

For an undirected graph, MAKE-SET every vertex and UNION(u,v) for each arriving edge. The invariant is that the DSU partition equals the connected components of the processed-edge subgraph. Initially both consist of singletons. An edge within a component changes neither structure; an edge between components joins exactly those two. Induction proves that at termination FIND(u)=FIND(v) exactly when an undirected u–v path exists.

On a..f, insert ab, cd, bc, ab, ef, de. The partitions are {ab},{c},{d},{e},{f}; {ab},{cd},{e},{f}; {abcd},{e},{f}; the same after repeated ab; {abcd},{ef}; and {abcdef}. Five merges succeed during six UNION calls.

Why union maintains exactly the connected components (continued)

Edge stream: each partition change

Each row shows the partition after an edge. Repeated edge 4 leaves the set count fixed.

			Set count
ab	⇒	{a,b} {c} {d} {e} {f}	5
cd	⇒	{a,b} {c,d} {e} {f}	4
bc	⇒	{a,b,c,d} {e} {f}	3
ab	⇒	{a,b,c,d} {e} {f}	3
ef	⇒	{a,b,c,d} {e,f}	2
de	⇒	{a,b,c,d,e,f}	1

Five merges: 6→5→4→3→3→2→1. All six UNION calls still check representatives.

Static DFS or BFS already finds components in $\Theta(n+|E|)$. DSU additionally supports connectivity queries as edges arrive and Kruskal cycle tests. Ignoring directions in a directed graph gives weak connectivity, not strongly connected components.

21.2 Linked-list representation of disjoint sets

Set records and member back-pointers

Each set record stores head, tail, and size; each member stores next and a set pointer. The representative is record.head. An empty collection takes O(1) to initialize. MAKE-SET creates next=NIL, head=tail=x, size=1, and x.set pointing to the new record. FIND returns x.set.head in O(1).

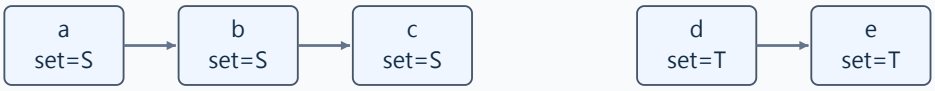
UNION reads S=x.set and T=y.set. If equal, return the head. Swap S,T if S.size<T.size; on ties keep the first S. Set S.tail.next=T.head, S.tail=T.tail, and S.size+=T.size. Traverse the old T list, redirecting each member's set pointer to S, then discard T's record. The traversal retains the original next links, so successors are not lost. Tail splicing alone does not update representatives.

One UNION costs $\Theta(1+\min(|S|,|T|))$. Merging a→b→c with d→e produces a→b→c→d→e. Only d,e change set records; the tail moves from c to e and size from 3 to 5. Their FIND calls now return a.

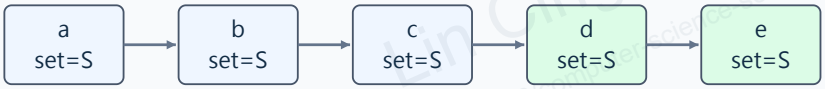
21.2 Linked-list representation of disjoint sets (continued)

Weighted lists: update set as well as next

Before: S head=a, tail=c, size 3; T head=d, tail=e, size 2.



Append short to long. Change only d,e set pointers to S, and set c.next=d.



After: S head=a, tail=e, size 5. Green members changed set; arrows show next.

A quadratic counterexample and the doubling proof

Suppose the second set is always appended to the first. Starting with n singletons, UNION(x_2,x_1), UNION(x_3,x_2), … makes each new singleton absorb the old long list. Pointer rewrites number 1,2,…,n−1, totaling n(n−1)/2. Thus m=2n−1 calls cost $\Theta(n^2)$, despite constant-time next splicing with a tail pointer.

Weighted union moves only the shorter set. Each rewrite of a fixed member x at least doubles its set size. After k moves, size is at least 2^k and at most n, so $k \leq \text{floor}(\log_2 n)$. Summing over n members gives at most n floor($\log_2 n$) rewrites, plus O(m) call overhead, for O(m+n log n). Use log(n+1) for small n. A single merge of two n/2-member lists still costs Θ(n); the sequence bound does not give worst-case O(log n) per union.

For $n=2^k$, repeatedly pair equal-sized sets. Each of k rounds rewrites exactly n/2 pointers, totaling $(n/2)\log_2 n$. Constant-time MAKE and FIND are consistent with this total union cost.

Accounting for eight elements

Keep the first set on equal sizes. Use this sequence again for the forest. Letters are handles, not priority keys.

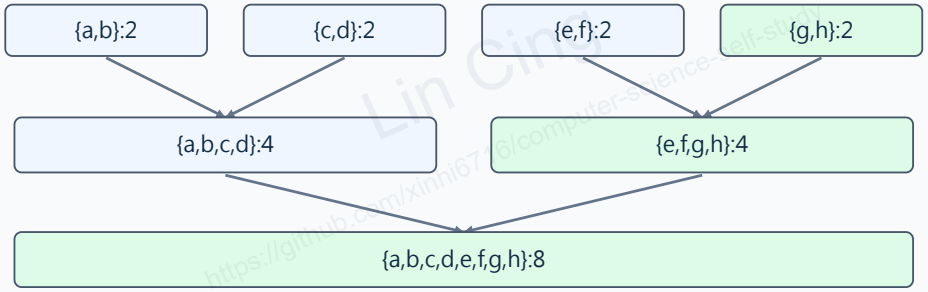
UNION	Old sizes	Members whose set changes	New representative/size	Cumulative rewrites
a,b	1,1	b	a／2	1
c,d	1,1	d	c／2	2
e,f	1,1	f	e／2	3
g,h	1,1	h	g／2	4
a,c	2,2	c,d	a／4	6
e,g	2,2	g,h	e／4	8
a,e	4,4	e,f,g,h	a／8	12

Accounting for eight elements (continued)

For h, the set pointer follows h→g→e→a, and the containing size grows 2→4→8. It changes three times; a never changes. Total rewrites are 12, not three for every element. In the earlier singleton-absorption sequence, weighting reduces the total to n−1 rewrites.

Equal-size pairwise merges: three rounds of rewrites

Each round rewrites four set pointers. Green marks h's containing set: 2→4→8.



Total rewrites 4+4+4=12; h changes 3 times, a changes 0 times. Sets never split.

21.3 Disjoint-set forests

Self-parent roots and the LINK precondition

Each element stores parent and rank; a root points to itself. MAKE-SET sets parent[x]=x and rank[x]=0. UNION must first find both roots. Linking arbitrary x directly to y can split an existing set or create a cycle.

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    if r==s: return r
    return LINK(r,s)
LINK(r,s):                    // r and s are distinct roots
    if rank[r]<rank[s]: swap r,s
    parent[s]=r
    if rank[r]==rank[s]: rank[r]+=1
    return r
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21.3 Disjoint-set forests (continued)

After linking roots, all old members still reach the new root. One root loses its self-parent, with no reverse path creating a cycle. Unequal ranks leave the retained rank unchanged; equal ranks increase it by one. Rank forests and size-weighted lists are different representations. Rank is not exact set size.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two-pass FIND preserves the old parent before overwriting

```
FIND(x):
    r=x
    while parent[r]!=r: r=parent[r]
    while parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    return r
```

The first pass finds the root without mutation. The second follows the old route and redirects every parent to r. Overwriting parent[x] before saving it jumps immediately to r and misses intermediate nodes. Recursive parent[x]=FIND(parent[x]) finds the root upward and redirects while returning. Its stack uses at most the path height; the two-pass iterative version uses O(1) extra space.

Compression reconnects nodes only to the root of the same tree, preserving roots and the partition. It never changes rank. Visiting s nodes costs Θ(s). Forest storage is O(n). Member lists can support enumeration, but parent pointers alone do not automatically provide O(set size) enumeration.

The same eight elements before and after compression

For the seven earlier unions, equal ranks retain the first root. The successive root ranks are a1,c1,e1,g1,a2,e2,a3. Final parents are b→a, c→a, d→c, e→a, f→e, g→e, h→g; a points to itself.

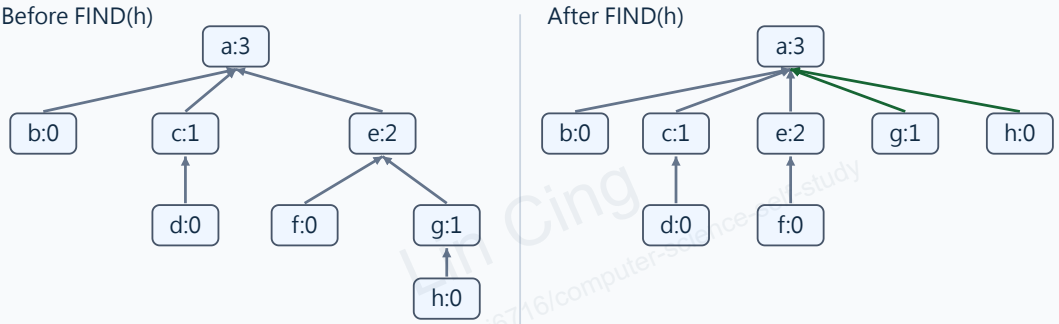
Vertex	a	b	c	d	e	f	g	h
rank	3	0	1	0	2	0	1	0
Parent before FIND(h)	a	a	a	c	a	e	e	g
Parent after FIND(h)	a	a	a	c	a	e	a	a

The first pass takes h→g→e→a, three links. The second sets the parents of h,g,e to a; e already had that value. Nodes d→c and f→e are outside this path and do not change. All ranks remain fixed. A later FIND(h) takes one link, but FIND(d) still takes two. One find does not flatten the whole tree.

The same eight elements before and after compression (continued)

The same eight-node tree: parents before and after compression

Nodes show key:rank; arrows show parent; root self-loops omitted; green links changed.



h→g→e→a; only h,g links change. Node e already points to a; d,f are off-path.

Rank bounds height and historical membership

Induct on operations to prove that a root's current height is at most its rank. A singleton has height zero. For unequal ranks, the smaller height plus one is at most the larger rank; equal ranks allow one extra level. Compression only shortens paths. A rank-r root contains at least 2^r members: its rank rises only by merging two rank-(r−1) sets, each of size at least 2^{r-1}. Other merges only add members. A nonroot keeps its rank and the historical membership bound from when it was last a root. Thus every rank is at most floor($\log_2 n$).

Rank is not a lower bound on current height. After compressing every element in the example, a retains rank 3 but has height 1. A rank-2 root can likewise be the center of a four-node star. Rank is not current subtree size either, since compression can move descendants away without reducing rank.

Rank alone gives O(log n) per FIND and O(m log n) overall. Pairwise equal-rank merges create height $\log_2 n$, and repeated uncompressed finds on a deepest leaf attain that order. Without rank, arbitrary links can create a linear chain and a first FIND of Θ(n). That counterexample does not apply when both heuristics are present.

21.4 Analysis of union by rank with path compression

Ackermann iteration means composition

Fix final n and α(n) throughout the sequence. Write $f^{(0)}(j) = j$, $f^{(i+1)}(j) = f(f^{(i)}(j))$. For k≥0,j≥1, define $A_0(j) = j + 1$, $A_k(j) = A_{k-1}^{(j+1)}(j)$. The superscript counts **function compositions**.

From $A_0^{(i)}(j) = j + i$, we obtain $A_1(j) = 2j + 1$. Induction gives $A_1^{(i)}(j) = 2^i(j + 1) - 1$; taking i=j+1 yields $A_2(j) = 2^{j+1}(j + 1) - 1$. Hence $A_3(1)=A_2(A_2(1))=A_2(7)=2047$.

Monotonicity also follows inductively. A_0 is strictly increasing and $A_0(j)\geq j+1$. If A_{k-1} has both properties, starting from a larger j and iterating more times gives a larger result, so A_k is strictly increasing in j. Since j+1≥2, at least two iterations give $A_k(j)>A_{k-1}(j)$, and $A_k(j)\geq j+1$. These monotonicities justify the level bounds and good-node inequality below.

The chosen inverse-Ackermann convention

Use the single-parameter convention of CLRS third edition, Section 21.4. For `k>=0`, `j>=1`, define

- `A_0(j)=j+1`
- `A_k(j)=A_{k-1}^{(j+1)}(j)`, with `j+1` compositions of `A_{k-1}`.
- `alpha(n)=min{k>=0 | A_k(1)>=n}`.

This differs from common two-parameter conventions. Values are `A_0(1)=2`, `A_1(1)=3`, `A_2(1)=7`, `A_3(1)=2047`, `A_4(1)>10^80`. Thus `alpha(2)=0`, `alpha(8)=3`, and `alpha(2048)=4`.

Keep the small-n cases in the complete theorem.

For `m` operations including `n` MAKE-SET calls, total cost is `O(m(1+alpha(n)))`.

Only for `n>=3`, where `alpha(n)>=1`, can this simplify to `O(m alpha(n))`. Applying that shorter expression to n=0,1,2 would incorrectly give zero work.

The chosen inverse-Ackermann convention (continued)

Ackermann thresholds and fixed-scale potential accounting

Fix final n; $\alpha=\min\{k\geq 0: A_k(1)\geq n\}$; $\Psi=C\Phi$ with one fixed $C>c>0$.

Thresholds $A_k(1)$	One Ψ pays for all three operations
$A_0(1)=2$	MAKE: new rank 0, $\Delta\Psi=0$; cost $O(1)$.
$A_1(1)=3$	LINK: $\Delta\Psi\leq C\alpha$; actual cost $O(1)$.
$A_2(1)=7$	FIND: s nodes; $g\geq\max(0,s-(\alpha+2))$.
$A_3(1)=2047$	$\Delta\Psi\leq -Cg$; $cs-Cg\leq\max(c,C)(\alpha+2)$ 。
$A_4(1)>10^{80}$	Initial $\Psi=0$, final $\Psi\geq 0\Rightarrow\Sigma\text{actual}\leq\Sigma\text{amortized}$.

$\alpha(2)=0$, $\alpha(8)=3$, $\alpha(2048)=4$. Complete bound: $O(m(1+\alpha(n)))$.

Rank and path invariants

MAKE-SET creates a rank-zero singleton. LINK puts the lower-rank root under the higher-rank root; equal ranks increase the retained rank by one. Path compression changes parents, never ranks.

Rank monotonicity. Roots use `x.parent=x`. An implementation using NIL can express the same invariant with an equivalent root test. Induction on operations proves

- a root points to itself and increases rank only when equal-rank roots merge; MAKE-SET starts at rank zero;
- a nonroot satisfies `x.rank < x.parent.rank`;
- once a node becomes a nonroot, its rank stays fixed, while parent rank can only increase.

Ranks therefore increase strictly along every path toward a root.

Rank range. A root x represents at least `2^{x.rank}` members. Merging rank-r roots combines disjoint sets of at least `2^r+2^r=2^{r+1}` members. Unequal-rank merges do not reduce the larger set. A nonroot retains the historical lower bound from losing root status. Thus `rank(x)<=floor(log_2 n)<=n-1`.

Rank and path invariants (continued)

For a nonroot, `level(x)` depends on its parent's rank and never decreases. However, `iter(x)` can reset and decrease when the level rises. They are not both individually monotone.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Levels, iteration counts, and potential

Let `alpha=alpha(n)`. For a nonroot of rank `r=x.rank>=1`, define

- `level(x)=max{k | x.parent.rank>=A_k(r)}`
- `iter(x)=max{i | x.parent.rank>=A_level(x)^(i)(r)}`

Because `A_0(r)=r+1` and the parent rank is at least r+1, `level(x)>=0`. For `n>=3`, `A_alpha(1)>=n>x.parent.rank`, while `A_alpha(r)>=A_alpha(1)`. Hence `level(x)<alpha`.

The inequality `iter(x)>=1` follows from `x.parent.rank>=A_level(x)(r)`. If `iter(x)>=r+1`, then `x.parent.rank>=A_level(x)^(r+1)(r)=A_{level(x)+1}(r)`,

contradicting maximality of `level(x)`. Therefore `1<=iter(x)<=r`.

Define each node's potential and sum `Phi=sum phi(x)`.

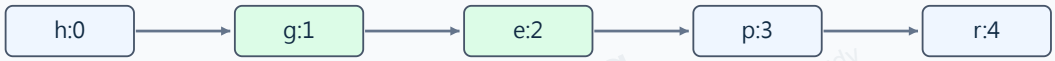
```
phi(x) = alpha * r                                  if x is a root or r=0
phi(x) = (alpha-level(x))*r - iter(x)                if x is a nonroot and r>=1
```

Levels, iteration counts, and potential (continued)

The first branch multiplies `alpha` by `r`. A rank-zero MAKE-SET node has potential `alpha*0=0`. In the second branch, `0<=level<alpha` and `1<=iter<=r` imply `0<=phi(x)<alpha*r`.

FIND values: progress through level and iter

n=16, α=3; after FIND(h), h,g,e,p all point to r. All ranks stay fixed.



Before (level,iter)				
—	(0,1)	(0,1)	(0,1)	root
After (level,iter)				
—	(1,1)	(0,2)	(0,1)	root
φ 0→0	φ 2→1	φ 5→4	φ 8→8	φ 12→12

Good nodes g,e: path potential 27→25. Exceptions: rank 0, root, last node at each level.

A rising level can reset the iteration count

Fix rank r. When level stays fixed, increasing parent rank can only increase the number of iterations allowed at that level, so iter cannot decrease in this case.

If level rises from k to k'>k, iter can fall from i to i', even to 1. The potential difference remains

```
phi_after-phi_before
= -(k'-k)r + (i-i').
```

Since $k'-k \geq 1$, $i' \geq 1$, and $i \leq r$, we have $i-i' \leq r-1$. The difference is at most $-r+(r-1)=-1$. Thus larger level, followed by larger iter within a fixed level, progresses lexicographically and decreases phi by at least one.

This potential handles both increases of iter within a level and resets caused by a level increase. No global monotonicity of iter is needed or valid.

The good-node inequality for FIND

Fix one compression path. A node x of rank `r>=1` is good if some later nonroot y on that path has `level(x)=level(y)=k`.

Before compression, the definitions and rank monotonicity give

```
rank(y)>=rank(parent(x))
rank(parent(y))>=A_k(rank(y))
>=A_k(rank(parent(x)))
>=A_k^(iter(x)+1)(rank(x)).
```

After compression, x and y share one root, so

`new parent(x).rank = new parent(y).rank >= old parent(y).rank >= A_k^(iter(x)+1)(rank(x)).`

If this new bound does not raise the level, iter(x) increases by at least one. An iteration count exceeding r would contradict `iter<=rank`, and therefore forces a level increase. Every good node progresses lexicographically in `(level,iter)`, decreasing phi by at least one.

Charging FIND with fixed constants

A find path has s nodes. At most `alpha+2` are exceptions: a rank-zero starting node if present, the root, and the last path node at each level `0,...,alpha-1`. Hence there are at least

`g=max(0,s-(alpha+2))`.

Every good node decreases Phi by at least one, and other path nodes do not increase it. If actual scanning cost is at most `c s`, fix a scaled potential `Psi=C Phi` with `C>c`. Scaling by one fixed positive constant preserves the previous inequalities, and each good node decreases Psi by at least C. Thus `Delta Psi<=-Cg`, giving

```
amortized FIND = actual+Delta Psi
<= c*s-C*g
<= max(c,C)(alpha+2).
```

This proves `O(alpha+2)=O(1+alpha)` using explicit constants. An expression `O(s)-Omega(s-alpha)` cannot simply cancel without matching those constants.

Charging FIND with fixed constants (continued)

MAKE-SET costs O(1) and adds zero potential. LINK costs O(1). The attached former root does not increase potential. Existing children of the retained root do not increase potential because their parent rank cannot fall. Only the retained root can increase rank, by one after an equal-rank merge, adding at most alpha to Phi and `c alpha` to Psi. Thus amortized LINK costs `O(1)+C alpha=O(1+alpha)`, while MAKE-SET costs O(1).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Total cost and the scope of the bound

Expand UNION into two FIND calls and at most one LINK. This multiplies the operation count by at most three. Let actual cost be `c_i` and amortized cost

`a_i=c_i+Delta Psi_i`.

Summing telescopes.

```
sum_i a_i
= sum_i c_i + Psi_final-Psi_initial.
```

Therefore

`sum_i c_i = sum_i a_i-Psi_final+Psi_initial`.

The initial structure is empty, so `Psi_initial=0`. Every node potential is nonnegative, so `Psi_final>=0`. Hence `Psi_initial-Psi_final<=0`, and

`sum_i c_i<=sum_i a_i`.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Total cost and the scope of the bound (continued)

This turns per-operation amortized bounds into a bound on actual sequence cost. Applying the FIND and LINK bounds to the expanded sequence gives

$O(m(1+\alpha(n)))$.

and the original sequence has the same asymptotic bound. For $n \geq 3$, simplify to $O(m\ \alpha(n))$; for n=0,1,2, retain the constant-work case with alpha=0.

The theorem bounds the worst total cost of an operation sequence. It does not give worst-case O(alpha(n)) per FIND. Union by rank bounds a single path by O(log n), so a linear chain is impossible under that hypothesis. Removing rank permits linear chains but changes the theorem's assumptions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 21.

Cormen et al., *Introduction to Algorithms*, 3rd ed., § 21.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study